

Trabalho 2 - Sistemas Operacionais (INF1316)

Gerencia de Memoria - Simulador de Memoria Virtual (sim-virtual)

1. Identificacao

Nome: Caio Faria Brito Martins Chaves Matricula: 2410162

Nome: Lucas Hufnagel Gromann de Araujo Goes Matricula: 2410845

Algoritmos implementados pelo grupo: LRU, NRU, Relogio e Otimo.

2. Resumo do projeto

O sim-virtual e um simulador de memoria virtual com paginacao, escrito em linguagem C. Ele recebe na linha de comando o algoritmo de substituicao, o arquivo .log com a sequencia de acessos a memoria, o tamanho da pagina (em KB) e o tamanho da memoria fisica (em MB). Cada linha do arquivo .log traz um endereco logico de 32 bits em hexadecimal seguido de R (leitura) ou W (escrita). Para cada acesso o simulador atualiza os bits de controle, detecta as faltas de pagina e, quando a memoria esta cheia, simula a substituicao de uma pagina por outra. No final imprime a configuracao usada, o numero de faltas de pagina e o numero de paginas sujas que precisariam ser escritas no disco.

O indice da pagina e obtido descartando os s bits menos significativos do endereco ($page = addr \gg s$). Como a pagina e potencia de 2, o valor de s e o numero de vezes que ela pode ser dividida por 2: para 4 KB temos $s = 12$ e para 8 KB, $s = 13$. O numero de quadros da memoria fisica e a memoria dividida pelo tamanho da pagina; por exemplo, 1 MB com paginas de 4 KB da 256 quadros.

Estrutura geral: o programa esta dividido em dois arquivos (sim-virtual.c e sim-virtual.h), com um Makefile e um script run_all.sh que roda todas as combinacoes de uma vez. O laco principal usa um contador de tempo, incrementado a cada acesso, para registrar o momento de cada referencia. As faltas de pagina sao contadas toda vez que uma pagina e carregada (inclusive no preenchimento inicial dos quadros) e as escritas sao contadas somente quando uma pagina modificada e removida. Paginas sujas que ainda estao na memoria no fim da execucao nao sao contadas, conforme o enunciado.

Estruturas de dados relevantes:

- Quadro: representa um quadro da memoria fisica. Guarda a pagina que esta nele (ou -1 se vazio), o bit R (referenciada), o bit M (modificada/suja) e o instante do ultimo acesso. Esses campos atendem a todos os algoritmos.
- Tabela de paginas: vetor indexado pelo numero da pagina que diz em qual quadro a pagina esta, ou -1 se ela nao esta na memoria. Permite saber em tempo constante se uma pagina esta residente. Como os enderecos tem 32 bits ela pode ter muitas entradas (ate cerca de 1 milhao para 4 KB), mas como simulamos um unico programa isso nao e problema.
- Vetor de acessos: a sequencia inteira do .log carregada em memoria. E indispensavel para o algoritmo Otimo, que precisa olhar os acessos futuros, e e usado de forma uniforme pelos demais.

Estruturas especificas de cada algoritmo de substituicao:

- LRU: usa o campo de instante do ultimo acesso de cada quadro. Na substituicao escolhe o quadro com o menor instante (a pagina ha mais tempo sem uso).
- NRU: usa os bits R e M para classificar cada quadro em quatro classes - 0:(R=0,M=0), 1:(R=0,M=1), 2:(R=1,M=0) e 3:(R=1,M=1) - e remove uma pagina da menor classe nao vazia. O bit R e zerado periodicamente (a cada 1000 acessos), imitando o tick do relógio do sistema operacional.

- Relógio (segunda chance): trata os quadros como uma lista circular com um ponteiro. Se a página apontada tem $R=1$, ela ganha uma segunda chance (zera R e o ponteiro avança); se tem $R=0$, é a escolhida. O ponteiro é guardado entre as substituições.

- Ótimo: para cada página são pre-calculadas, em ordem, as posições (instantes) em que ela é acessada, guardadas em um único vetor com início/fim por página. Assim o próximo uso de qualquer página residente sai em tempo constante, e a vítima é a página com o próximo uso mais distante.

3. Pseudo-código do algoritmo Ótimo

```
funcao Otimo(acessos A[0..n-1], num_quadros):
    para i de 0 ate n-1:                                # ocorrencias de cada pagina
        p = pagina(A[i])
        guarda i na lista de ocorrencias de p

    quadros = {}
    faltas = 0
    escritas = 0
    para tempo de 0 ate n-1:
        p = pagina(A[tempo])
        escrita = (A[tempo] e 'W')
        avanca a lista de p                             # passa a apontar o proximo uso de p

        se p ja esta nos quadros:
            marca R (e M, se escrita) de p             # acerto, nao e falta
            continua

        faltas = faltas + 1                             # page fault
        se ha quadro livre:
            carrega p em um quadro livre
        senao:
            vitima = pagina com o MAIOR proximo uso    # vazia = nunca mais usada
            se a vitima esta suja (M=1):
                escritas = escritas + 1
            remove a vitima e carrega p no lugar dela
    retorna (faltas, escritas)
```

4. Dados de cada simulacao

Cada um dos quatro programas (1.000.000 de acessos cada) foi simulado para todas as combinacoes de algoritmo, tamanho de pagina (4 e 8 KB) e tamanho de memoria (1, 2 e 4 MB), num total de 96 simulacoes. PF = numero de page-faults; PS = numero de paginas sujas escritas. Os dados foram gerados pelo script run_all.sh (resultados.csv).

Programa: compilador.log

Algoritmo	Pagina	Memoria	PF	PS
-----	-----	-----	-----	-----
LRU	4 KB	1 MB	25308	4231
LRU	4 KB	2 MB	10425	2173
LRU	4 KB	4 MB	4391	955
LRU	8 KB	1 MB	36707	5775
LRU	8 KB	2 MB	21091	3839
LRU	8 KB	4 MB	7632	1756
NRU	4 KB	1 MB	41819	3379
NRU	4 KB	2 MB	38178	2282
NRU	4 KB	4 MB	22559	239
NRU	8 KB	1 MB	41975	4464
NRU	8 KB	2 MB	38805	3313
NRU	8 KB	4 MB	32604	1687
Relogio	4 KB	1 MB	26789	4528
Relogio	4 KB	2 MB	11472	2398
Relogio	4 KB	4 MB	4765	1086
Relogio	8 KB	1 MB	38827	6250
Relogio	8 KB	2 MB	22655	4162
Relogio	8 KB	4 MB	8412	1981
Otimo	4 KB	1 MB	12667	2483
Otimo	4 KB	2 MB	5662	1328
Otimo	4 KB	4 MB	3047	726
Otimo	8 KB	1 MB	21271	3711
Otimo	8 KB	2 MB	10118	2190
Otimo	8 KB	4 MB	4232	1078

Programa: matriz.log

Algoritmo	Pagina	Memoria	PF	PS
-----	-----	-----	-----	-----
LRU	4 KB	1 MB	5673	1866
LRU	4 KB	2 MB	3632	1159
LRU	4 KB	4 MB	2803	789
LRU	8 KB	1 MB	10928	3273
LRU	8 KB	2 MB	4745	1623
LRU	8 KB	4 MB	2950	985
NRU	4 KB	1 MB	16410	1901
NRU	4 KB	2 MB	13863	1283
NRU	4 KB	4 MB	9133	276
NRU	8 KB	1 MB	17373	2470
NRU	8 KB	2 MB	14764	1652
NRU	8 KB	4 MB	12175	1072
Relogio	4 KB	1 MB	5980	1973
Relogio	4 KB	2 MB	3795	1218
Relogio	4 KB	4 MB	2901	809
Relogio	8 KB	1 MB	12896	3844

Relogio	8 KB	2 MB	4870	1676
Relogio	8 KB	4 MB	3064	1011
Otimo	4 KB	1 MB	3572	1267
Otimo	4 KB	2 MB	2672	985
Otimo	4 KB	4 MB	2543	672
Otimo	8 KB	1 MB	5361	1827
Otimo	8 KB	2 MB	2969	1109
Otimo	8 KB	4 MB	2295	843

Programa: compressor.log

Algoritmo	Pagina	Memoria	PF	PS
-----	-----	-----	-----	-----
LRU	4 KB	1 MB	397	48
LRU	4 KB	2 MB	317	0
LRU	4 KB	4 MB	317	0
LRU	8 KB	1 MB	533	132
LRU	8 KB	2 MB	255	0
LRU	8 KB	4 MB	255	0
NRU	4 KB	1 MB	595	0
NRU	4 KB	2 MB	317	0
NRU	4 KB	4 MB	317	0
NRU	8 KB	1 MB	916	18
NRU	8 KB	2 MB	255	0
NRU	8 KB	4 MB	255	0
Relogio	4 KB	1 MB	432	71
Relogio	4 KB	2 MB	317	0
Relogio	4 KB	4 MB	317	0
Relogio	8 KB	1 MB	577	163
Relogio	8 KB	2 MB	255	0
Relogio	8 KB	4 MB	255	0
Otimo	4 KB	1 MB	317	36
Otimo	4 KB	2 MB	317	0
Otimo	4 KB	4 MB	317	0
Otimo	8 KB	1 MB	345	86
Otimo	8 KB	2 MB	255	0
Otimo	8 KB	4 MB	255	0

Programa: simulador.log

Algoritmo	Pagina	Memoria	PF	PS
-----	-----	-----	-----	-----
LRU	4 KB	1 MB	11240	4092
LRU	4 KB	2 MB	5823	2444
LRU	4 KB	4 MB	4468	1846
LRU	8 KB	1 MB	16479	5546
LRU	8 KB	2 MB	8556	3395
LRU	8 KB	4 MB	4732	2094
NRU	4 KB	1 MB	25906	5127
NRU	4 KB	2 MB	24154	4385
NRU	4 KB	4 MB	18277	2146
NRU	8 KB	1 MB	24885	6002
NRU	8 KB	2 MB	22747	4616
NRU	8 KB	4 MB	20944	3952
Relogio	4 KB	1 MB	11876	4319
Relogio	4 KB	2 MB	6119	2587
Relogio	4 KB	4 MB	4618	1937

Relógio	8 KB	1 MB	17923	6070
Relógio	8 KB	2 MB	9140	3666
Relógio	8 KB	4 MB	4875	2171
Otimo	4 KB	1 MB	5981	2575
Otimo	4 KB	2 MB	4125	1884
Otimo	4 KB	4 MB	3890	1565
Otimo	8 KB	1 MB	9179	3528
Otimo	8 KB	2 MB	4748	2160
Otimo	8 KB	4 MB	3498	1620

5. Analise de desempenho

Comparacao entre os algoritmos. Em todas as 96 simulacoes o algoritmo Otimo teve o menor numero de page-faults, como esperado, ja que ele e o limite inferior teorico. No compilador com pagina de 4 KB e memoria de 1 MB, por exemplo, o Otimo teve 12.667 faltas, contra 25.308 do LRU, 26.789 do Relogio e 41.819 do NRU - quase metade das faltas do LRU. Entre os algoritmos realizaveis, o LRU e o Relogio ficaram sempre muito proximos, com o Relogio um pouco pior, o que faz sentido porque o Relogio e justamente uma aproximacao do LRU usando so o bit R. O NRU foi o que mais gerou faltas, pois separa as paginas em apenas quatro classes e depende do momento em que os bits R sao zerados, sendo o mais grosseiro dos quatro. Em compensacao, o NRU foi o que menos escreveu paginas sujas, porque prefere remover paginas nao modificadas: no compilador com 4 KB e 4 MB ele escreveu apenas 239 paginas, contra 955 do LRU e 1.086 do Relogio.

Memoria fixa, variando o tamanho da pagina. Mantendo a memoria fixa e dobrando a pagina (de 4 para 8 KB), o numero de quadros cai pela metade, mas cada pagina cobre o dobro do espaco de enderecos. Para o compilador, o matriz e o simulador, o numero de page-faults aumentou com paginas de 8 KB: no matriz com LRU e 1 MB as faltas passaram de 5.673 (4 KB) para 10.928 (8 KB), e no compilador de 25.308 para 36.707. Nesses casos perder metade dos quadros pesou mais do que o ganho de cobertura, ou seja, a localidade espacial nao foi forte o suficiente para compensar. Ja o compressor tem um conjunto de trabalho pequeno, que cabe na memoria a partir de 2 MB; ali as faltas passam a ser so as obrigatorias e, como paginas maiores significam menos paginas distintas, o tamanho de 8 KB gerou menos faltas (255 contra 317, em 2 e em 4 MB). Quanto as escritas, paginas maiores tendem a aumentar o numero de paginas sujas removidas, porque uma pagina maior tem mais chance de conter algum endereco escrito (no compilador com LRU e 4 MB as escritas subiram de 955 para 1.756).

Variando o tamanho da memoria. Aumentar a memoria fisica sempre reduziu o numero de page-faults, em todos os algoritmos e programas. No compilador com LRU e pagina de 4 KB as faltas cairam de 25.308 (1 MB) para 10.425 (2 MB) e 4.391 (4 MB). Quando o conjunto de trabalho inteiro passa a caber na memoria (como no compressor a partir de 2 MB), sobram apenas as faltas obrigatorias e o numero de escritas vai a zero, ja que nenhuma pagina precisa mais ser removida.

6. Conclusao

Os resultados ficaram de acordo com a teoria: o Otimo e melhor que o LRU e o Relogio, que por sua vez sao melhores que o NRU em numero de faltas; o NRU compensa escrevendo menos paginas no disco; e mais memoria sempre ajuda. Entre os algoritmos realizaveis, o LRU foi o melhor na maioria dos cenarios, com o Relogio como alternativa mais simples de implementar e desempenho praticamente igual. A comparacao tambem mostrou que aumentar o tamanho da pagina nem sempre ajuda: quando o conjunto de trabalho nao cabe na memoria, ter menos quadros acaba prejudicando o desempenho. Para validar a implementacao, as saidas dos quatro algoritmos foram comparadas com uma implementacao de referencia feita a parte, e os numeros coincidiram em todos os casos testados.

7. Como compilar e executar

make

./sim-virtual LRU compilador.log 8 2

para rodar todas as combinacoes de uma vez (com os .log no diretorio):

./run_all.sh # gera o arquivo resultados.csv